

Wo Modell und Provider geändert werden

Anleitung für das RAG-Projekt: an welchen Stellen das Chat-Modell geändert werden kann und an welchen der Provider (OpenRouter vs. OpenAI).

Verwandte Dateien:

modellauflösung.txt - vollständiger Workflow der Modellauflösung
provider_aufloesung.txt - vollständiger Workflow der Provider-Auflösung

Die Regel, die beides verbindet

Das Präfix in der Modellkennung setzt den Provider mit.

CHAT_MODEL=openai:gpt-4o-mini -> Provider wird OPENAI
CHAT_MODEL=openrouter:z-ai/glm-5.3 -> Provider wird OPENROUTER

Darum gibt es für den Provider kein eigenes CLI-Flag: wer den Provider für einen einzelnen Lauf wechselt, tut das über das Präfix.

Prioritäten

Modell: Argument > CHAT_MODEL aus .env > Katalog-Default
(default=True)
Provider: Präfix > CHAT_PROVIDER aus .env > Feld-Default (OPENROUTER)

Modell ändern – vier Stellen

Ebene	Ort	Wirkung	Beispiel
-----	-----	-----	
Grundeinstellung default=True auf	models.py	wenn nichts anderes gesetzt ist	Flag einen anderen
Eintrag	default=True		
Projektkonfig CHAT_MODEL=openrouter:z-ai/glm-5.3	.env CHAT_MODEL	bei jedem Start	
Einzellauf (Code) ChatBot(model="openrouter: deepseek/deepseek-v4-flash")	ChatBot(model=...)	nur dieser Bot	
Einzellauf (CLI) rag_project.main_chatbot openrouter:z-ai/glm-5.3	--model ...	nur dieser Aufruf	uv run python -m --model

Leer lassen ist eine gültige Wahl: CHAT_MODEL= gilt als "nicht gesetzt", und der Katalog-Default greift. Dafür sorgt der Validator _blank_model_to_none in config.py.

Verfügbare Modelle

```
uv run rag-project --models
```

Für den Chatbot besonders relevant (nach Eignung):

Kennung	Typ	Wofür
-----	-----	
openrouter:openai/gpt-5.6-luna	balanced	Default, Allround-Wahl
openrouter:z-ai/glm-5.3	balanced	gleichwertige
Alternative		
openrouter:anthropic/claude-sonnet-5	flagship	stärkste Zitat-Treue,
teurer		
openrouter:deepseek/deepseek-v4-flash	fast	günstig für Testläufe
openrouter:ibm-granite/granite-4.2-8b	open_weight	offenes Modell
openai:gpt-4o-mini	fast	OpenAI-Default

Provider ändern – drei Stellen

Ebene	Ort	Wirkung	Beispiel
-----	-----	-----	
Grundeinstellung	config.py	wenn CHAT_PROVIDER	chat_provider:
ChatProvider	Feld-Default	fehlt	=
ChatProvider.OPENROUTER			
Projektkonfig	.env	bei jedem Start	
CHAT_PROVIDER=openai	CHAT_PROVIDER		
Einzellauf	Präfix in der	nur dieser Aufruf	--model
openai:gpt-4o-mini	Modellkennung		

Für den Provider gibt es KEINE vierte Ebene – kein eigenes CLI-Flag.
Der Präfix-Weg deckt den Einzellauf ab.

Die vier praktischen Fälle

FALL 1 – dauerhaft OpenRouter, anderes Modell

Nur eine Zeile in der .env:

```
CHAT_PROVIDER=openrouter
CHAT_MODEL=openrouter:z-ai/glm-5.3
```

FALL 2 – dauerhaft auf OpenAI umstellen

CHAT_PROVIDER ändern, CHAT_MODEL leer lassen, OpenAI-Key hinterlegen:

```
CHAT_PROVIDER=openai
CHAT_MODEL=
OPENAI_API_KEY=sk-...
```

Ergebnis: openai:gpt-4o-mini als Katalog-Default.

FALL 3 – nur für einen Testlauf zu OpenAI

Die .env bleibt unangetastet:

```
uv run python -m rag_project.main_chatbot --model openai:gpt-4o-mini
```

WICHTIG: Das Präfix setzt auch den Provider. Der Aufruf läuft über OpenAI, obwohl CHAT_PROVIDER=openrouter in der .env steht – und verlangt deshalb den OPENAI_API_KEY. Fehlt er:

```
OPENAI_API_KEY is not set
```

obwohl die .env auf OpenRouter zeigt. Das ist die einzige Stelle im Projekt, an der die .env den Provider nicht bestimmt.

FALL 4 – Modell ohne Präfix

Funktioniert, wenn der Name beim aktiven Provider eindeutig ist:

```
CHAT_PROVIDER=openrouter
CHAT_MODEL=z-ai/glm-5.3
```

Sucht zuerst openrouter:z-ai/glm-5.3, findet es, fertig.
Findet es nichts, sucht der Resolver im Gesamtkatalog und ÜBERNIMMT DEN PROVIDER, bei dem das Modell liegt – der aktive Provider kann auf diesem Weg kippen:

```
CHAT_PROVIDER=openai
CHAT_MODEL=z-ai/glm-5.3
-> erst openai:z-ai/glm-5.3 -> nicht gefunden
-> Fallback: liegt bei OpenRouter -> Provider kippt auf OPENROUTER
```

Der Sonderfall im Katalog

Ein einziger Name existiert bei beiden Providern:

```
openrouter:openai/gpt-5.6-luna
openai:gpt-5.6-luna
```

Ohne Präfix ist die Kennung nicht auflösbar:

CHAT_MODEL=gpt-5.6-luna	-> UnknownModelError
CHAT_MODEL=openrouter:gpt-5.6-luna	-> eindeutig
CHAT_MODEL=openai:gpt-5.6-luna	-> eindeutig

Der Fehler nennt beide qualifizierten Formen als Vorschlag. Dies ist der einzige Fall im Katalog, in dem das Präfix zwingend nötig ist.

Was der Provider steuert

Ein Provider-Wechsel ändert drei Dinge gleichzeitig:

	OpenRouter	OpenAI
base_url	https://openrouter.ai/api/v1	
https://api.openai.com/v1		
geprüfter Key	OPENROUTER_API_KEY	OPENAI_API_KEY
Fehlertext	OPENROUTER_API_KEY is not set	OPENAI_API_KEY is not set

Technisch läuft beides über DIESELBE KLASSE ChatOpenAI – es gibt keine ChatOpenRouter-Klasse im Projekt. OpenRouter bietet eine OpenAI-kompatible API an; der Unterschied sind nur diese drei Werte plus zwei optionale Herkunfts-Kopfzeilen (HTTP-Referer, X-Title), die nur bei OpenRouter gesetzt werden.

Im Code liefert der Provider diese Werte über Properties:

```
class ChatProvider(str, Enum):
    OPENAI = "openai"
    OPENROUTER = "openrouter"

    @property
    def base_url(self) -> str: ...
    @property
    def key_field(self) -> str: ...
    @property
    def env_var(self) -> str: ...
```

Prüfen, was gerade aktiv ist

```
# Aktive Konfiguration: Provider, base_url, Modell, geprüfter Key
# (Keys nur als Fingerabdruck, nie im Klartext)
uv run rag-project --show-config

# Katalog mit * am Default je Provider
```

```
uv run rag-project --models
```

```
# Auflösung inkl. Präfix-Vorrang
uv run python -c "
from rag_project.providers import resolve_target
print('Praefix openai ->',
resolve_target('openai:gpt-4o-mini').provider.value)
print('Praefix openrouter->',
resolve_target('openrouter:z-ai/glm-5.3').provider.value)
print('ohne Praefix ->', resolve_target().provider.value)
"

# Welche Keys sind gesetzt?
uv run python -c "
from rag_project.config import get_settings
s = get_settings()
print('openai_api_key gesetzt :', s.openai_api_key is not None)
print('openrouter_api_key gesetzt:', s.openrouter_api_key is not None)
"
```

Kurzreferenz

Modell dauerhaft ändern	-> .env: CHAT_MODEL
Modell einmalig ändern	-> --model <kennung> oder ChatBot(model=...)
Modell-Grundeinstellung	-> models.py: default=True verschieben
Provider dauerhaft ändern	-> .env: CHAT_PROVIDER
Provider einmalig ändern	-> Präfix: --model openai:gpt-4o-mini
Provider-Grundeinstellung	-> config.py: Feld chat_provider
Reihenfolge	-> Präfix > .env > Default